

Actividad 2: Implementación de Role-Based Access Control (RBAC)

Tema: Control de acceso basado en roles

Objetivo: Implementar RBAC en una API Flask



¿Qué es RBAC?

RBAC (Role-Based Access Control) asigna permisos a los usuarios según su **rol** (*admin, usuario, invitado, etc.*).

Configurar RBAC en Flask

Flask es un framework escrito en Python que permite **crear aplicaciones web rápidamente** y con un mínimo número de líneas de código. Está basado en la especificación WSGI de Werkzeug y el motor de templates Jinja2 y tiene una licencia BSD. <https://flask.palletsprojects.com/en/stable/>

Instalar Flask y Flask-Principal en un entorno virtual

1. Crear un entorno virtual en el directorio actual:

```
python3 -m venv rbac
```

2. Activar el entorno virtual:

```
source rbac/bin/activate
```

```
(root@kali)~/home/kali
# tree rbac
rbac
├── bin
│   ├── activate
│   ├── activate.csh
│   ├── activate.fish
│   ├── Activate.ps1
│   ├── flask
│   ├── pip
│   ├── pip3
│   ├── pip3.13
│   ├── python → python3
│   ├── python3 → /usr/bin/python3
│   └── python3.13 → python3
├── include
│   └── python3.13
├── lib
│   └── python3.13
│       ├── site-packages
│       └── blinker
│           ├── base.py
│           ├── init__.py
│           ├── pycache
│           ├── base.cpython-313.pyc
│           ├── __init__.cpython-313.pyc
│           └── _utilities.cpython-313.pyc
```

3. Instalar los paquetes dentro del entorno virtual:

```
pip install flask flask-principal flask-login
```

4. Crear el archivo **rbac_api.py**:

```
mousepad rbac_api.py
```

```
from flask import Flask, request, jsonify, make_response
from flask_principal import Principal, Permission, RoleNeed, Identity, AnonymousIdentity,
identity_changed, identity_loaded
from flask_login import LoginManager, UserMixin, login_user, logout_user, current_user

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Configurar Flask-Principal
principals = Principal(app)

# Configurar Flask-Login
login_manager = LoginManager()
login_manager.init_app(app)

# Definir roles y permisos
admin_permission = Permission(RoleNeed('admin'))
user_permission = Permission(RoleNeed('user'))

# Base de datos simulada (puedes cambiarlo por una real)
db_users = {
    "admin": {"password": "admin123", "role": "admin"},
    "user": {"password": "user123", "role": "user"}
}

# Modelo de usuario
class User(UserMixin):
    def __init__(self, username, role):
        self.id = username
        self.role = role

@login_manager.user_loader
def load_user(user_id):
    user_data = db_users.get(user_id)
    if user_data:
        return User(user_id, user_data['role'])
    return None

@app.route('/login', methods=['POST'])
def login():
    data = request.json
    username = data.get("username")
    password = data.get("password")
    user_data = db_users.get(username)
    if user_data and user_data['password'] == password:
        user = User(username, user_data['role'])
        login_user(user)
        identity_changed.send(app, identity=Identity(user.id))
        return jsonify({"message": "Inicio de sesión exitoso"})
    return jsonify({"message": "Credenciales incorrectas"}), 401
```

```
@app.route('/logout')
def logout():
    logout_user()
    identity_changed.send(app, identity=AnonymousIdentity())

    #Invalidar la cookie de sesión
    response = make_response(jsonify({"message": "Sesión cerrada"}))
    response.set_cookie('session', '', expires=0) # Eliminar la cookie de sesión
    return response

@identity_loaded.connect_via(app)
def on_identity_loaded(sender, identity):
    if current_user.is_authenticated:
        identity.user = current_user
        identity.provides.add(RoleNeed(current_user.role))

@app.route('/admin')
@admin_permission.require(http_exception=403)
def admin_panel():
    return jsonify({"message": "Bienvenido al panel de administración"})

@app.route('/user')
@user_permission.require(http_exception=403)
def user_dashboard():
    return jsonify({"message": "Bienvenido al dashboard de usuario"})

if __name__ == '__main__':
    app.run(debug=True)
```

5. Cuando se finaliza, se puede salir del entorno virtual con:

```
deactivate
```

Probar la implementación de RBAC en Flask

1. Instalar dependencias

Antes de ejecutar la API, asegurarse de tener instaladas las bibliotecas necesarias dentro del entorno creado anteriormente:

```
pip install flask flask-principal flask-login
```

2. Ejecutar la API

Ejecutar el script *rbac_api.py* en la terminal o entorno de desarrollo:

```
python rbac_api.py
```

La API se ejecutará en *http://127.0.0.1:5000/*

```
(rbac)-(root@kali)-[/home/kali]
# python rbac_api.py
* Serving Flask app 'rbac_api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 645-236-326
```

3. Probar las rutas con *Postman* o *cURL*

Iniciar sesión

Realizar una solicitud POST a */login*

- **Usuario Admin:**

```
{
  "username": "admin",
  "password": "admin123"
}
```

- **Usuario Regular:**

```
{
  "username": "user",
  "password": "user123"
}
```

* Usando *cURL*

```
curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d '{"username": "admin", "password": "admin123"}' -c cookies.txt
```

Esto guardará la sesión en *cookies.txt*.

```
(root@kali)-[/home/kali]
# curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d '{"username": "admin", "password": "admin123"}' -c cookies.txt
{"message": "Inicio de sesi\u00f3n exitoso"}
# cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_127.0.0.1 FALSE / FALSE 0 session.eJxNjssKgZAP8l51LWJGsSf0Y2-0DBStF4kNJ_r9CL15k5zMeNtuk-uaFthz7c0IsbHBD1zGYQazU0GEAsIEe2LKMeR2nDLUQSkpg64qRUGXK00PVXkgLx0Kjikj1ksy8R_EdKkx11UCNIWGOQkmunPftAAc0SM5NGd40cu27_G5LxvF5oFl3b3M4nHW0a2_LWN6zHstzMrf_-ALFMRN8.Z8bTgw.otJY2xhvU_iaGiJht2I8PM-qQmQ
```

```
(root@kali)-[/home/kali]
# curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d '{"username": "admin", "password": "admin1"}' -c cookies.txt
{"message": "Credenciales incorrectas"}

(root@kali)-[/home/kali]
# cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

(root@kali)-[/home/kali]
```

Acceder a rutas protegidas

Probar acceso al panel de administración (/admin)

```
curl -X GET http://127.0.0.1:5000/admin -b cookies.txt
```

- Si **admin**, se recibirá:

```
{ "message": "Bienvenido al panel de administración" }
```
- Si **usuario regular**, se recibirá un **403 Forbidden**.

```
(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/admin -b cookies.txt
<!doctype html>
<html lang=en>
<title>403 Forbidden</title>
<h1>Forbidden</h1>
<p>&lt;Permission needs={Need(method=&#39;role&#39;;, value=&#39;admin&#39;)} excludes=set()&gt;</p>

(root@kali)-[/home/kali]
# curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d '{"username": "admin", "password": "admin123"}' -c cookies.txt
{"message": "Inicio de sesi\u00f3n exitoso"}

(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/admin -b cookies.txt
{"message": "Bienvenido al panel de administraci\u00f3n"}

(root@kali)-[/home/kali]
```

Probar acceso al dashboard de usuario (/user)

```
curl -X GET http://127.0.0.1:5000/user -b cookies.txt
```

- Si **usuario o admin**, se recibirá:

```
{ "message": "Bienvenido al dashboard de usuario" }
```

- Si no se ha iniciado sesión, se recibirá un **403 Forbidden**.

```
(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/user -b cookies.txt
<!doctype html>
<html lang=en>
<title>403 Forbidden</title>
<h1>Forbidden</h1>
<p>&lt;Permission needs={Need(method=&#39;role&#39;, value=&#39;user&#39;)} excludes=set()&gt;</p>

(root@kali)-[/home/kali]
# curl -X POST http://127.0.0.1:5000/login -H "Content-Type: application/json" -d '{"username": "user", "password": "user123"}' -c cookies.txt
{
  "message": "Inicio de sesi\u00f3n exitoso"
}

(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/user -b cookies.txt
{
  "message": "Bienvenido al dashboard de usuario"
}

(root@kali)-[/home/kali]
#
```

Cerrar sesión

```
curl -X GET http://127.0.0.1:5000/logout -b cookies.txt -c cookies.txt
```

Esto eliminará la sesión y se necesitará volver a iniciar sesión.

- **-b cookies.txt** → Usa el archivo *cookies.txt* para enviar cookies almacenadas previamente (es decir, mantiene la sesión activa si hay una cookie válida).
- **-c cookies.txt** → Guarda las cookies recibidas en *cookies.txt* (esto es útil para actualizar el archivo con una cookie expirada o eliminada después de cerrar sesión).

```
(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/logout -b cookies.txt -c cookies.txt
{
  "message": "Sesi\u00f3n cerrada"
}

(root@kali)-[/home/kali]
# cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

#HttpOnly_127.0.0.1 FALSE / FALSE 0 session .eJYrVspMSc0rySyp1EssLcmIL6ksSFWyyivNydF
ByGSmQIRqAZu4EWc.Z8bVdA.0XJe2fQ_sVSZBhTKq_wHdenVSv0

(root@kali)-[/home/kali]
# curl -X GET http://127.0.0.1:5000/user -b cookies.txt
<!doctype html>
<html lang=en>
<title>403 Forbidden</title>
<h1>Forbidden</h1>
<p>&lt;Permission needs={Need(method=&#39;role&#39;, value=&#39;user&#39;)} excludes=set()&gt;</p>

(root@kali)-[/home/kali]
#
```

Realizar las siguientes pruebas de *seguridad*:

- Intentar acceder a */admin sin autenticación*.
- Iniciae sesión como *usuario* y probar a acceder a */admin*.
- Probar a *cerrar sesión* y acceder nuevamente a */user* o */admin*.

Mejores prácticas

- Nunca asignar permisos directamente a usuarios, ya que dificulta la gestión, el mantenimiento y el control de accesos. En su lugar, se deben asignar permisos a roles, y luego vincular usuarios a estos roles.
 - o *Facilita la administración*: Cambiar los permisos de un rol automáticamente afecta a todos los usuarios del rol.
 - o *Reduce errores humanos*: Evita olvidar revocar permisos cuando un usuario cambia de función o deja la organización.
 - o *Mejora la seguridad*: Los permisos son controlados a nivel de rol, lo que minimiza accesos no autorizados.
 - o *Cumple con normativas* como ISO 27001, GDPR, PCI-DSS, que exigen control y trazabilidad de accesos.
- Implementar el Principio de Privilegio Mínimo (PoLP - Principle of Least Privilege) es una práctica de seguridad que establece que los usuarios, aplicaciones y sistemas solo deben tener los permisos estrictamente necesarios para realizar sus funciones, reduciendo así los riesgos de accesos indebidos y ataques.
 - o *Reduce la superficie de ataque*: Minimiza el impacto de ataques en caso de vulnerabilidades.
 - o *Limita el daño en caso de compromiso*: Si una cuenta es hackeada, el atacante tendrá acceso restringido.
 - o *Previene errores humanos*: Evita cambios o eliminaciones accidentales de archivos o configuraciones críticas.
 - o *Cumple con normativas de seguridad*: Requerido en estándares como ISO 27001, NIST, PCI-DSS y GDPR.
- Usar una base de datos real (SQLite, PostgreSQL, MySQL). Para proyectos que requieren almacenamiento persistente y estructurado, se debe usar una base de datos real como SQLite, PostgreSQL o MySQL, en lugar de almacenar datos en archivos planos, variables en memoria o bases de datos en memoria como H2.
 - o *Persistencia de datos*: La información no se pierde al reiniciar la aplicación.
 - o *Consultas eficientes*: Uso de SQL para recuperar datos de forma rápida y estructurada.
 - o *Escalabilidad*: PostgreSQL y MySQL permiten manejar grandes volúmenes de datos y múltiples conexiones.
 - o *Integridad y seguridad*: Soporte para transacciones, autenticación y cifrado.
 - o *Soporte de concurrencia*: Permiten múltiples usuarios y procesos simultáneos sin corrupción de datos.